The Metatheory of Crush's Lowering Pipeline

lean-crush

August 2026

Abstract

Crush translates a supported finite collection of Lean propositions into SMT commands. This document explains the current mathematical framework for that translation: intrinsically typed higher-order syntax, datatype-aware source models, total flattened defunctionalization, untyped SMT command semantics, SMT datatype declarations, representation of source values by an existing target type plus a predicate, and exact agreement between the formal encoding and the SMT commands emitted by the Crush translator. It is written in dependency order: each object and notation is introduced before the first theorem that uses it. The final theorem justifies the reified higher-order theory and the exact semantic command set produced by the modeled lowering pipeline; it does not assume that an external solver is correct and does not assign a denotation to arbitrary `Lean.Expr`.

Contents

1 Reader's guide

Contents of this section. The question answered by the formalization; proof stages; notation and scope.

1.1 What is being proved

The formalization studies this pipeline:

supported Lean fact array ⟶ reified, intrinsically typed HO theory ⟶ combined flattened FO theory ⟶ untyped SMT command sequence

The first arrow is a partial recognizer: every accepted fact is reified under one common signature and datatype environment, and unsupported Lean syntax is rejected. The remaining formal transformations are total. SMT datatype commands and recursive well-formedness definitions are modeled components of the final command set. The principal semantic conclusion is:

> If the emitted command sequence represents the formal encoding and is semantically unsatisfiable, then the reified higher-order source theory is unsatisfiable in every source model satisfying the free-datatype model condition.

The proof is by countermodel construction. Starting from one source model that satisfies both the free-datatype equations for every reified datatype block and the complete higher-order theory, Lean constructs one model of all flattened sentences, closure equations, and extensionality formulas, then constructs one SMT model satisfying SMT datatype declarations, derived recursive guards, ordinary declarations, and assertions. Contraposition yields the desired reflection of unsatisfiability.

1.2 Proof stages

The argument is organized into seven stages:

1. Define an intrinsically typed higher-order source language and its ordinary function-space semantics.

2. Describe productive monomorphic datatype blocks, their finite constructor-tree semantics, and the free-datatype source-model contract.

3. Define an intrinsically typed first-order target whose function-value sorts are opaque, then give total flattened defunctionalization.

4. Interpret every translated sentence in one canonical model and prove simultaneous validity of the aggregate target theory.

5. Give semantics to the modeled untyped SMT command fragment and prove that the generated command sequence has exactly the intended models.

6. Represent source values inside target types using predicates, validate SMT datatype and recursive guard commands, and combine all components in one SMT model.

7. Retain structural Lean reification and compare the SMT commands emitted by the Crush translator with the commands generated by the formal encoding.

A running example begins in Example 2.3 after the source syntax has been defined. Each later section revisits the same sentence using only the objects introduced by that point in the document.

1.3 Metavariables and notation

The following metavariables are used consistently throughout the document. Semantic and translation notation is introduced later, beside the definition that gives it meaning.

e, f, a	source or target terms; f is usually function-valued,
φ, ψ	Boolean formulas; a closed formula is a sentence,
Φ	a finite higher-order source theory,
β	an opaque source base-sort name,
σ, τ, υ	source types,
κ	a first-order sort,
Γ, Δ	typing contexts,
Σ	a typed signature,
T	a finite first-order target theory,
M, N	semantic models,
r	a typed renaming,
ρ	a source valuation,
ν	a target valuation,
η	an untyped SMT evaluation environment,
v, w	individual semantic values,
$\mathcal{E}$	a concrete FO-to-SMT encoding choice,
$\mathcal{G}$	a guard-aware SMT encoding choice,
C	an ordered SMT command sequence (implemented as a Lean array).

Subscripts s, t, and q mean source, typed target, and untyped SMT model, respectively. Proof-facing Lean declarations use the corresponding Greek binders for valuations and Roman r for renamings; executable allocation and state code uses descriptive names when several objects have the same role.

We reserve $\Phi \vdash \varphi$ strictly for syntactic entailment in an explicitly specified deductive system. The present formalization defines no such proof calculus: intrinsically typed syntax is written as inhabitation of an indexed term family, and semantic claims use satisfaction. In particular, $\vdash$ does not denote typing, context lookup, or membership in a datatype block.

2 The intrinsically typed higher-order source

Contents of this section. Types, contexts, terms, and ordinary higher-order semantics.

2.1 Types, contexts, and signatures

Definition 2.1 (Source types)**.** *Let β range over opaque base-sort names. Source types are generated by*

$$\tau ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \tau \to \sigma.$$

Arrows are nondependent and associate to the right. A context $\Gamma = [\tau_0, \ldots, \tau_{n-1}]$ records local binders with the newest binder at the head. We write Γ, τ for extension by a new binder of type τ; this is stored as $\tau :: \Gamma$ in Lean. A signature Σ is a list of the types of global constants. [1]

[1] Lean formalization: `Crush/Metatheory/HO/Core.lean`.

We write $\Gamma \ni \tau$ for the intrinsically typed lookup judgment. Its inhabitants are de Bruijn references, generated by:

$$\frac{}{Z : \Gamma, \tau \ni \tau} \qquad \frac{i : \Gamma \ni \tau}{S\, i : \Gamma, \sigma \ni \tau}\ .$$

The judgment is intrinsically witnessed: its derivation simultaneously records the position and proves the type at that position. The witness Z denotes index zero and $S\, i$ denotes the successor of the index denoted by i. Forgetting the derivation yields the natural-number index used when collecting free variables. Constants use the same typed-reference representation in Σ. Therefore a variable or constant with the wrong type cannot be constructed.

For $i : \Gamma \ni \tau$, write x_i for the local-variable term selected by i. For $k : \Sigma \ni \tau$, write c_k for the global-constant term selected by k from the signature. The subscripts are typed lookup witnesses, not variable or constant names.

Definition 2.2 (Source terms)**.** *An intrinsically typed source term is written* $e : \mathsf{Term}_\Sigma(\Gamma, \tau)$*. This indexed family has variables, constants, Boolean literals and connectives, same-type equality, lambda, application, and universal and existential quantification:*

$$\frac{i : \Gamma \ni \tau}{x_i : \mathsf{Term}_\Sigma(\Gamma, \tau)} \qquad \frac{k : \Sigma \ni \tau}{c_k : \mathsf{Term}_\Sigma(\Gamma, \tau)}$$

$$\frac{e : \mathsf{Term}_\Sigma(\Gamma, \tau, \sigma)}{\lambda x{:}\tau.e : \mathsf{Term}_\Sigma(\Gamma, \tau \to \sigma)} \qquad \frac{f : \mathsf{Term}_\Sigma(\Gamma, \tau \to \sigma) \qquad a : \mathsf{Term}_\Sigma(\Gamma, \tau)}{f\, a : \mathsf{Term}_\Sigma(\Gamma, \sigma)}$$

$$\frac{e_1 : \mathsf{Term}_\Sigma(\Gamma, \tau) \qquad e_2 : \mathsf{Term}_\Sigma(\Gamma, \tau)}{e_1 = e_2 : \mathsf{Term}_\Sigma(\Gamma, \mathsf{Bool})} \qquad \frac{\varphi : \mathsf{Term}_\Sigma(\Gamma, \tau, \mathsf{Bool}) \qquad Q \in \{\forall, \exists\}}{Qx{:}\tau.\varphi : \mathsf{Term}_\Sigma(\Gamma, \mathsf{Bool})}.$$

The omitted rules are the standard typed Boolean rules. A formula is a term of type Bool*; a sentence is a formula in the empty context.* [2]

2.2 Running example: captured functions over trees

Example 2.3 (The source formula used throughout). Fix an opaque base-sort identity β_{Tree} and abbreviate

$$\mathsf{T} = \mathsf{Base}(\beta_{\mathsf{Tree}}), \qquad \mathsf{F} = \mathsf{T} \to \mathsf{T}.$$

For a local variable $f : \mathsf{F}$, define the following metanotation for a source term:

$$\mathsf{square}(f) = \lambda x{:}\mathsf{T}.\, f\, (f\, x).$$

This is not a new constant in Σ: it abbreviates a lambda that captures the surrounding variable f. Our running sentence is

$$\varphi_{\mathsf{sq}} = \forall f{:}\mathsf{F}.\, \forall g{:}\mathsf{F}.\ \big(\big(\forall x{:}\mathsf{T}.\ f\, x = g\, x\big) \to \mathsf{square}(f) = \mathsf{square}(g)\big)\,. \tag{RE}$$

It deliberately contains the three cases that make defunctionalization interesting: higher-order quantification over f and g, complete applications such as $f\, (f\, x)$, and equality between function values. In ordinary source semantics it is valid by function extensionality. Later sections show exactly which first-order sorts, application symbols, closure symbols, equations, extensionality formulas, and SMT commands it generates.

[2] Lean formalization: `Crush/Metatheory/HO/Core.lean`.

At this point T is merely an opaque base type. Section 3 gives β_{Tree} the meaning of a custom datatype; the HO term grammar itself does not change.

Quantifiers may range over arrow types. Thus the modeled language is genuinely higher-order even though its arrows are nondependent.

2.3 Source models

Definition 2.4 (Source type denotation)**.** *A carrier family B assigns a nonempty type $B(\beta)$ to each opaque base sort. Its extension to source types is*

$$[\![\mathsf{Bool}]\!]_B = \mathsf{Prop}, \qquad [\![\mathsf{Base}(\beta)]\!]_B = B(\beta), \qquad [\![\tau \to \sigma]\!]_B = [\![\tau]\!]_B \to [\![\sigma]\!]_B.$$

Thus the only choices in B are the carriers assigned to opaque base sorts.

Definition 2.5 (Source model and valuation)**.** *A source model M_s consists of a carrier family B and an interpretation $c_k^{M_s} \in [\![\tau]\!]_B$ for each $k : \Sigma \ni \tau$. A valuation ρ for Γ assigns a value in $[\![\tau]\!]_B$ to every $i : \Gamma \ni \tau$. We write $\rho[v]$ for extension by a new head value v.*

Definition 2.6 (Source denotation)**.** *Denotation is structural. Variables use ρ, constants use M_s, and the Boolean forms have their ordinary meanings. From this point onward,*

$$[\![e]\!]_{M_s,\rho}$$

denotes the value of e in M_s under ρ.[3] *The higher-order clauses are*

$$\begin{aligned}
[\![\lambda x{:}\tau.e]\!]_{M_s,\rho} &= \lambda v.[\![e]\!]_{M_s,\rho[v]},\\
[\![f\ a]\!]_{M_s,\rho} &= [\![f]\!]_{M_s,\rho}([\![a]\!]_{M_s,\rho}),\\
[\![\forall x{:}\tau.\varphi]\!]_{M_s,\rho} &= \forall v \in [\![\tau]\!]_B, [\![\varphi]\!]_{M_s,\rho[v]},\\
[\![\exists x{:}\tau.\varphi]\!]_{M_s,\rho} &= \exists v \in [\![\tau]\!]_B, [\![\varphi]\!]_{M_s,\rho[v]}.
\end{aligned}$$

Definition 2.7 (Sentences, theories, and satisfaction)**.** *A source sentence is a Boolean term in the empty context. A finite source theory is a list $\Phi = [\varphi_1, \ldots, \varphi_n]$ of sentences sharing one signature Σ. We introduce the satisfaction notation*

$$\begin{aligned}
M_s \models \varphi \quad &\iff \quad [\![\varphi]\!]_{M_s,\varnothing},\\
M_s \models^{\mathsf{T}} \Phi \quad &\iff \quad \forall \varphi \in \Phi.\ M_s \models \varphi,
\end{aligned}$$

where $\varnothing$ is the unique valuation of the empty context. [4]

Definition 2.8 (Source unsatisfiability)**.** *Sentence and theory unsatisfiability are semantic properties:*

$$\begin{aligned}
\mathsf{Unsat}_{\mathsf{HO}}(\varphi) \quad &\iff \quad \forall M_s.\ \neg(M_s \models \varphi),\\
\mathsf{Unsat}_{\mathsf{HO}}(\Phi) \quad &\iff \quad \forall M_s.\ \neg(M_s \models^{\mathsf{T}} \Phi).
\end{aligned}$$

For a proof-relevant model contract $\mathcal{L}(M_s)$, the restricted form is

$$\mathsf{Unsat}^{\mathcal{L}}_{\mathsf{HO}}(\Phi) \quad \iff \quad \forall M_s.\ \forall \ell : \mathcal{L}(M_s).\ \neg(M_s \models^{\mathsf{T}} \Phi).$$

The Lean definitions are `Unsatisfiable`*,* `TheoryUnsatisfiable`*, and* `UnsatisfiableUnder`*.* [5]

[3] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.
[4] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.
[5] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.

3 Datatype-aware source semantics

Contents of this section. Typed mutual blocks; their embedding as HO base sorts and constants; finite constructor trees; free-datatype source models.

Custom datatype support strengthens the class of source models without changing the HO term language. Constructors, selectors, and testers remain ordinary constants in Σ; a separate typed description says which constants they are and what laws their interpretations obey.

3.1 Typed datatype blocks

Definition 3.1 (Mutual datatype block)**.** *Fix an arity $n > 0$. A field sort is either an external base sort or a direct reference to one of the n datatypes:*

$$q ::= \mathsf{base}(\beta) \mid \mathsf{data}(d), \qquad d \in \mathsf{Fin}(n).$$

A field declaration pairs a descriptive name with q; a constructor is a name and an ordered field list; a datatype declaration is a base-sort identity and an ordered constructor list. A block B assigns one datatype declaration to every $d \in \mathsf{Fin}(n)$. Typed positions, rather than names, identify members. We write $\beta_{B,d}$ for the base-sort identity stored in the declaration at position d. The corresponding typed reference types are

$$\mathsf{DataRef}(B), \qquad \mathsf{CtorRef}(B, d, K), \qquad \mathsf{FieldRef}(K, F).$$

Their inhabitants select, respectively, a datatype position d in B, a constructor K of d, and a field F of K. The notation deliberately mirrors the Lean type families `DataRef`, `CtorRef`, and `FieldRef`; none is an entailment relation. [6]

Definition 3.2 (Block well-formedness)**.** *A block is structurally well formed when it is nonempty, its datatype base-sort identities are pairwise distinct, and every field classified as external is fresh from those datatype identities. Direct recursive fields can refer only to the current block, so unsafe cross-block cycles are not representable. The executable predicate `wellFormed` is proved equivalent to this proposition.* [7]

3.2 Finite free values

Definition 3.3 (Free datatype algebra)**.** *For external carriers $B_0(\beta)$, the family $\mathsf{Val}_B(B_0, d)$ and its heterogeneous constructor arguments are defined mutually. It is the* free datatype algebra *generated by the constructors: every value has exactly one finite constructor tree*

$$v ::= K(a_1, \ldots, a_m),$$

where an external field contains an element of $B_0(\beta)$ and a recursive field contains another value $v' \in \mathsf{Val}_B(B_0, d')$. The height of a value is one plus the maximum height of its recursive fields. A block is productive *when every d has a finite value after all external carriers are replaced by the unit type. Here "free" means that constructor trees have no identifications or equations beyond those forced by their constructor and field structure; it does not mean that the formalization assumes an arbitrary user-supplied isomorphism between unrelated carriers.* [8]

[6] Lean formalization: `Crush/Metatheory/Datatype/Core.lean`.
[7] Lean formalization: `Crush/Metatheory/Datatype/Core.lean`.
[8] Lean formalization: `Crush/Metatheory/Datatype/Semantics.lean`.

Theorem 3.4 (Constructor, selector, and tester laws)**.** *Canonical values satisfy constructor injectivity and disjointness, constructor exhaustiveness, matching tester equations, matching selector equations, and strict height decrease along every recursive field. Selectors are total; on a nonmatching constructor their fallback is intentionally unconstrained.* [9]

3.3 Free-datatype source models

Definition 3.5 (Datatype symbol map)**.** *For a block B and source signature Σ, a symbol map $\mathsf{Symbols}(\Sigma, B)$ selects the ordinary HO constants used as each constructor, selector, and tester. The types of those constants are computed from the block: constructor fields form a curried arrow telescope, selectors map a datatype carrier to one field carrier, and testers return* Bool*.* [10]

Definition 3.6 (Custom datatypes in the HO language)**.** *The HO grammar has no separate datatype type former. The member d of a custom block B is represented by the ordinary source type*

$$\mathsf{Base}(\beta_{B,d}).$$

For a field sort q, define its HO type by

$$\mathsf{ty}_B(\mathsf{base}(\beta)) = \mathsf{Base}(\beta), \qquad \mathsf{ty}_B(\mathsf{data}(d')) = \mathsf{Base}(\beta_{B,d'}).$$

If constructor K of member d has fields $q_1, \ldots, q_m$, its symbol reference selects an ordinary constant of type

$$\mathsf{ty}_B(q_1) \to \cdots \to \mathsf{ty}_B(q_m) \to \mathsf{Base}(\beta_{B,d}).$$

A selector has type $\mathsf{Base}(\beta_{B,d}) \to \mathsf{ty}_B(q_i)$, and a tester has type $\mathsf{Base}(\beta_{B,d}) \to \mathsf{Bool}$. Consequently a datatype constructor, selector, or tester is not a new HO term form: it is an intrinsically typed constant application. A $\mathsf{Base}(\beta)$ not selected by a datatype environment remains completely opaque; membership in that environment and the source-model condition below are what give a selected base sort free-datatype meaning.

The datatype value type is therefore a base type, while constructor constants usually have function types because they accept fields and return that base type. This does not prevent an ordinary HO function from accepting or returning a datatype value. The supported datatype reifier does reject function-valued fields*: the modeled SMT datatype representation has only external base fields and recursive datatype fields, and the constructor-tree proof is stated for exactly those field sorts. Thus the datatype theorems assume, after all datatype parameters have been instantiated, that every stored field is either an external nonfunction base value or a direct recursive datatype value.* [11]

Example 3.7 (The running tree datatype). Give β_{Tree} from Example 2.3 one datatype declaration and fix a second, external base sort β_{Num}. In familiar notation the block is

$$\mathsf{Tree} ::= \mathsf{leaf}(\mathsf{value} : \mathsf{Num}) \mid \mathsf{node}(\mathsf{left} : \mathsf{Tree}, \mathsf{right} : \mathsf{Tree}), \qquad \mathsf{Num} = \mathsf{Base}(\beta_{\mathsf{Num}}).$$

Formally, the result type of both constructors is T, the leaf field has field sort $\mathsf{base}(\beta_{\mathsf{Num}})$, and the two node fields have field sort $\mathsf{data}(d_{\mathsf{Tree}})$. The symbol map selects ordinary HO constants with types

$$\mathsf{leaf} : \mathsf{Num} \to \mathsf{T}, \qquad \mathsf{node} : \mathsf{T} \to \mathsf{T} \to \mathsf{T},$$

[9] Lean formalization: `Crush/Metatheory/Datatype/Semantics.lean`.

[10] Lean formalization: `Crush/Metatheory/Datatype/Syntax.lean and Datatype/Model.lean`.

[11] Lean formalization: `Crush/Metatheory/Datatype/Syntax.lean, Datatype/Model.lean, and Datatype/Flattened.lean`.

together with the corresponding selectors and Boolean testers. The variables $f, g : \mathsf{F}$ in (RE) are ordinary HO functions over datatype values; they are permitted even though a function-valued *field* in the declaration above would be outside the modeled datatype fragment.

Example 3.8 (A tree that stores functions). Consider a parameterized Lean datatype with the familiar shape

$$\mathsf{Tree}(\alpha) ::= \mathsf{leaf}(\mathsf{value} : \alpha) \mid \mathsf{node}(\mathsf{left} : \mathsf{Tree}(\alpha), \mathsf{right} : \mathsf{Tree}(\alpha)).$$

Let $A = \mathsf{Base}(\beta_A)$. In the ground instance $\mathsf{Tree}(A \to A)$, the leaf constructor has type

$$(A \to A) \to \mathsf{Tree}(A \to A).$$

The problem is not that this constructor has an arrow type—every constructor that accepts a field does. The problem is that the value stored in its leaf is itself a function. Such an instance cannot supply a custom-datatype block in the present formalization, because its field is neither an external base value nor a recursive datatype value.

The HO language may still treat the whole type $\mathsf{Tree}(A \to A)$ as an opaque base type. It can then quantify over values of that type and form ordinary HO functions that accept or return them. That opaque interpretation provides no free-datatype laws for leaf and node: in particular, it does not provide constructor injectivity, disjointness, exhaustiveness, selector equations, or structural recursion.

Definition 3.9 (Free-datatype source model)**.** *Let $S : \mathsf{Symbols}(\Sigma, B)$. Lean calls this condition `IsFreeDatatypeModel`; we write it as* $\mathsf{FreeData}$*. A witness $\mathsf{FreeData}_B(S, M_s)$ contains an isomorphism*

$$M_s.B(\beta_{B,d}) \cong \mathsf{Val}_B(M_s.B, d)$$

for every datatype position d, identifies every selected constructor constant with its canonical curried interpretation, requires matching selectors to recover the selected field, and requires each tester to recognize exactly its constructor. Such a witness implies productivity of B. [12]

This is a semantic restriction on source interpretations, not a consequence of structural block well-formedness alone. In particular, the current lowering theorem assumes such a witness for each selected custom datatype; it does not claim that an arbitrary Lean declaration has already been proved isomorphic to the free algebra. The machine-checked tests construct the complete witness for a monomorphic option type with an external integer field, including its carrier isomorphism and constructor, selector, and tester equations. Thus the condition is inhabited at a nontrivial instance, while the general connection from Lean datatype declarations remains outside the theorem stated here. [13]

Definition 3.10 (Datatype environment)**.** *An environment $\mathcal{D} = [(B_1, S_1), \ldots, (B_k, S_k)]$ is an ordered list of blocks and their symbol maps in one shared signature. The judgment $\mathsf{FreeData}_{\mathcal{D}}(M_s)$ supplies a free-datatype witness for every entry. We write*

$$\mathsf{Unsat}^{\mathcal{D}}_{\mathsf{HO}}(\Phi) \iff \forall M_s.\ \forall \ell : \mathsf{FreeData}_{\mathcal{D}}(M_s).\ \neg(M_s \models^{\mathsf{T}} \Phi).$$

The empty environment recovers ordinary source unsatisfiability. [14]

[12] Lean formalization: `Crush/Metatheory/Datatype/Model.lean`.
[13] Machine-checked example: `Test/MetatheoryDatatype.lean`.
[14] Lean formalization: `Crush/Metatheory/Datatype/Model.lean`.

4 The intrinsically typed first-order target

Contents of this section. Erased sorts, abstract symbol families, target terms, and target models.

Definition 4.1 (First-order sorts and erasure)**.** *Target sorts are*

$$\kappa ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \mathsf{Fn}(\tau, \sigma).$$

Erasure is defined by

$$\lfloor \mathsf{Bool} \rfloor = \mathsf{Bool}, \qquad \lfloor \mathsf{Base}(\beta) \rfloor = \mathsf{Base}(\beta), \qquad \lfloor \tau \to \sigma \rfloor = \mathsf{Fn}(\tau, \sigma).$$

The last sort is opaque: it is not a function space in an arbitrary target model. Context erasure maps every entry pointwise. [15]

Definition 4.2 (Declarations and symbol families)**.** *A declaration*

$$d = (\kappa_1, \ldots, \kappa_n; \kappa)$$

records an ordered argument telescope $d.\mathsf{args} = [\kappa_1, \ldots, \kappa_n]$ *and a result sort* $d.\mathsf{result} = \kappa$*. Let* Decl *be the type of all such declarations. A* symbol family *is a declaration-indexed family of types*

$$S : \mathsf{Decl} \longrightarrow \mathbf{Type}, \qquad d \longmapsto S(d).$$

An inhabitant $p : S(d)$ *is one symbol identity whose declaration is exactly* d*. The fiber* $S(d)$ *may contain no symbols, one symbol, or several distinct symbols with the same argument and result sorts. Thus the declaration is part of a symbol's type, while its identity remains separate from its declaration.* [16]

Proposition 4.3 (FO image of datatype source symbols)**.** *Datatype membership does not change erasure: for every member* d *of* B*,*

$$\lfloor \mathsf{Base}(\beta_{B,d}) \rfloor = \mathsf{Base}(\beta_{B,d}).$$

The curried HO type of a selected constructor is flattened to one FO declaration

$$\big(\lfloor \mathsf{ty}_B(q_1) \rfloor, \ldots, \lfloor \mathsf{ty}_B(q_m) \rfloor; \mathsf{Base}(\beta_{B,d})\big).$$

Selected selectors and testers similarly receive declarations

$$d_{\mathsf{sel},i} = (\mathsf{Base}(\beta_{B,d}); \lfloor \mathsf{ty}_B(q_i) \rfloor),$$
$$d_{\mathsf{test}} = (\mathsf{Base}(\beta_{B,d}); \mathsf{Bool}).$$

The flattened symbol family retains these as the same source-constant identities selected by the datatype symbol map; only their types have been exposed as complete first-order argument telescopes. Thus a custom datatype is still a base carrier in FO, while its operations are ordinary, fully applied family symbols. [17]

[15] Lean formalization: `Crush/Metatheory/FO/Core.lean`.
[16] Lean formalization: `Crush/Metatheory/FO/Family.lean`.
[17] Lean formalization: `Crush/Metatheory/Datatype/Flattened.lean`.

Example 4.4 (The running datatype after erasure). The two source base types remain base sorts:

$$\lfloor \mathsf{T} \rfloor = \mathsf{Base}(\beta_{\mathsf{Tree}}), \qquad \lfloor \mathsf{Num} \rfloor = \mathsf{Base}(\beta_{\mathsf{Num}}),$$

while the type of a tree transformer becomes the opaque sort

$$\lfloor \mathsf{F} \rfloor = \mathsf{Fn}(\mathsf{T}, \mathsf{T}).$$

For readability, continue to write T for the erased tree base sort and F for this function-value sort. The datatype constructors have the first-order declarations

$$\mathsf{leaf} : (\mathsf{Num}; \mathsf{T}), \qquad \mathsf{node} : (\mathsf{T}, \mathsf{T}; \mathsf{T}).$$

Nothing in the arbitrary FO model yet says that the carrier of F contains actual functions. Defunctionalization supplies operations on that opaque carrier, and the soundness proof later chooses a canonical FO model in which its values really are source functions.

Definition 4.5 (Typed family arguments and symbol application)**.** *Terms and heterogeneous argument vectors are defined mutually. For a context Γ, the argument-vector family follows the declaration telescope:*

$$\begin{aligned} \mathsf{Args}_S(\Gamma, []) &= \{()\}, \\ \mathsf{Args}_S(\Gamma, \kappa_1 :: \bar{\kappa}) &= \{t \mid t : \mathsf{FamilyTerm}_S(\Gamma, \kappa_1)\} \times \mathsf{Args}_S(\Gamma, \bar{\kappa}). \end{aligned}$$

The only general symbol-application rule is

$$\frac{p : S(d) \qquad \bar{t} : \mathsf{Args}_S(\Gamma, d.\mathsf{args})}{p(\bar{t}) : \mathsf{FamilyTerm}_S(\Gamma, d.\mathsf{result})}.$$

Consequently the result sort and the complete ordered list of argument sorts are determined by the index d. A missing, extra, or incorrectly sorted argument cannot be represented. In particular, symbols are always fully applied; partial application is not a target-language operation.

For example, let $d = (\kappa_1, \kappa_2; \kappa)$ and let $p, q : S(d)$ be distinct symbols. Given

$$u : \mathsf{FamilyTerm}_S(\Gamma, \kappa_1) \qquad \text{and} \qquad v : \mathsf{FamilyTerm}_S(\Gamma, \kappa_2),$$

both $p(u, v)$ and $q(u, v)$ have sort κ, but they remain different terms. Neither $p(u)$ nor $p(v, u)$ is constructible.

Definition 4.6 (Target terms)**.** *For a symbol family S, the indexed type $t : \mathsf{FamilyTerm}_S(\Gamma, \kappa)$ contains variables, the fully applied family symbols above, Boolean literals and connectives, same-sort equality, and first-order quantifiers. It contains neither lambda nor a distinguished higher-order application constructor. A function value is merely an element of an opaque $\mathsf{Fn}(\tau, \sigma)$ carrier; applying that value later requires an ordinary family symbol with the appropriate declaration.* [18]

[18] Lean formalization: `Crush/Metatheory/FO/Family.lean and FO/FamilySemantics.lean`.

Why translation uses a family. The finite target syntax also exists: a finite signature is a list $\Xi = [d_0, \ldots, d_m]$, and a symbol occurrence carries a typed position showing that some d_j is its declaration. That form is useful after allocation, but inconvenient during recursive translation. A recursive branch may discover a new closure or application symbol; combining two branches then concatenates their declaration lists. If terms already stored numeric signature positions, those positions would have to be weakened and reindexed after every such concatenation.

With a family, the translator instead constructs a structural identity $p : S(d)$ immediately. Recursive results concatenate an ordered list of existential packages (d, p), including repeated uses, while already constructed terms remain unchanged. For flattened defunctionalization, S is an indexed inductive family whose constructors represent source constants, application symbols, and closures. Each constructor's result index is its exact declaration; the flattened symbol definition below spells out those shapes.

From abstract identities to concrete symbols. The abstraction is removed only at a boundary where allocation information is available. The library provides a total structural conversion to a finite signature Ξ. Write $\mathsf{Symbol}(\Xi, d)$ for the type of positions in Ξ that select declaration d. A typed resolver has one component

$$r_d : S(d) \longrightarrow \mathsf{Symbol}(\Xi, d) \qquad \text{for every } d \in \mathsf{Decl}.$$

The proved untyped SMT route can also encode family terms directly. Its encoding record similarly supplies one naming component

$$\mathsf{name}_d : S(d) \longrightarrow \mathsf{String} \qquad \text{for every } d \in \mathsf{Decl},$$

with injectivity both between declarations and between two inhabitants of the same fiber, plus freshness from logical built-ins. For each listed pair (d, p), the command encoder emits the argument and result sorts from d and the concrete identifier $\mathsf{name}(p)$. Thus delaying allocation loses neither typing nor symbol identity. [19]

Definition 4.7 (Target model)**.** *A target model M_t supplies a nonempty carrier $[\![\kappa]\!]_{M_t}$ for each target sort. A declaration denotes the curried semantic type*

$$[\![d]\!]_{M_t} = [\![\kappa_1]\!]_{M_t} \to \cdots \to [\![\kappa_n]\!]_{M_t} \to [\![\kappa]\!]_{M_t}$$

for $d = (\kappa_1, \ldots, \kappa_n; \kappa)$. The model then supplies an interpretation component for every declaration:

$$I^d_{M_t} : S(d) \longrightarrow [\![d]\!]_{M_t} \qquad \textit{for every } d \in \mathsf{Decl}.$$

Hence two inhabitants of the same $S(d)$ may receive different interpretations, while every interpretation necessarily has the type prescribed by d. Target term denotation feeds the denotations of the typed argument vector to $I^d_{M_t}(p)$. The satisfaction relations $M_t \models \varphi$ and $M_t \models^{\mathsf{T}} T$ are then the standard many-sorted first-order semantics. [20]

Definition 4.8 (Target-theory unsatisfiability)**.** *A typed first-order theory is semantically unsatisfiable when it has no target model:*

$$\mathsf{Unsat}_{\mathsf{FO}}(T) \quad \Longleftrightarrow \quad \forall M_t.\, \neg(M_t \models^{\mathsf{T}} T).$$

[19] Lean formalization: `Crush/Metatheory/FO/Family.lean, Defunctionalization/TermTranslation.lean, and SMT/Representation.lean`.

[20] Lean formalization: `Crush/Metatheory/FO/FamilySemantics.lean`.

5 Total flattened defunctionalization

Contents of this section. Arrow telescopes, closures, generated symbols, translation results, and the total algorithm.

5.1 Flattened arrow shape

Definition 5.1 (Arrow telescope)**.** *Every source type has a list of leading argument types* $\mathsf{args}(\tau)$ *and a non-arrow residual type* $\mathsf{res}(\tau)$*:*

$$\begin{aligned} \mathsf{args}(\mathsf{Bool}) &= [], & \mathsf{res}(\mathsf{Bool}) &= \mathsf{Bool}, \\ \mathsf{args}(\mathsf{Base}(\beta)) &= [], & \mathsf{res}(\mathsf{Base}(\beta)) &= \mathsf{Base}(\beta), \\ \mathsf{args}(\tau \to \sigma) &= \tau :: \mathsf{args}(\sigma), & \mathsf{res}(\tau \to \sigma) &= \mathsf{res}(\sigma). \end{aligned}$$

For example, if $\alpha = \tau_1 \to \tau_2 \to \upsilon$ *and* υ *is non-arrow, then* $\mathsf{args}(\alpha) = [\tau_1, \tau_2]$ *and* $\mathsf{res}(\alpha) = \upsilon$*.*

The indices of a target argument spine record that applying the listed arguments changes a source type α into a residual type γ. A *ground-result witness* proves that γ is Boolean or a base type; only then can the spine be emitted as one complete first-order symbol application. This indexed representation is what makes under-application impossible in the total translator. [21]

5.2 Captures, closures, and symbols

Definition 5.2 (Exact captures)**.** $\mathsf{FV}(e)$ *is the duplicate-free, left-to-right list of free de Bruijn positions in* e*. Beneath a binder, position zero is removed and positive positions are shifted down. A closure descriptor*

$$\chi = (\Gamma, \tau, \sigma, e), \qquad e : \mathsf{Term}_\Sigma(\Gamma, \tau, \sigma),$$

stores the lambda's context, domain, codomain, body, and the typed references selected from $\mathsf{FV}(e)$*. If their types are* $\gamma_1, \ldots, \gamma_m$*, these are the exact constructor captures.* [22]

Definition 5.3 (Flattened symbol family)**.** *The target symbol family has four semantic roles:*

1. *a source constant* $c : \alpha$*, declared with arguments* $\lfloor \mathsf{args}(\alpha) \rfloor$ *and result* $\lfloor \mathsf{res}(\alpha) \rfloor$*;*

2. *an application symbol for every arrow* $\tau \to \sigma$*, declared as*

$$\mathsf{app}_{\tau,\sigma} : \mathsf{Fn}(\tau, \sigma) \times \lfloor \mathsf{args}(\tau \to \sigma) \rfloor \longrightarrow \lfloor \mathsf{res}(\tau \to \sigma) \rfloor;$$

3. *a closure constructor*

$$\mathsf{clos}_\chi : \lfloor \gamma_1 \rfloor \times \cdots \times \lfloor \gamma_m \rfloor \longrightarrow \mathsf{Fn}(\tau, \sigma);$$

The roles remain distinct even when two declarations have the same shape. Certified translator hooks use the separate semantic contracts in `Hooks.lean`*; the total structural translator does not add a second symbol identity for them.* [23]

[21] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Spine.lean`.

[22] Lean formalization: `Crush/Metatheory/Defunctionalization/Collect.lean` and `Defunctionalization/Annotate.lean`.

[23] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Symbol.lean`.

5.3 What translation returns

Definition 5.4 (Term translation and auxiliary theory)**.** *For* $e : \mathsf{Term}_\Sigma(\Gamma, \tau)$*, write* $\mathcal{F}[\![e]\!]$ *for the total flattened translation result. It contains*

$$\mathcal{F}[\![e]\!] = (t_e, D_e, E_e, X_e),$$

where:

- t_e *is a target term in context* $\lfloor\Gamma\rfloor^\star$ *and sort* $\lfloor\tau\rfloor$*;*
- D_e *is the ordered list of required symbol declarations;*
- E_e *is the list of closure equations;*
- X_e *is the list of extensionality formulas;*

The generated auxiliary theory, the complete target theory of one sentence, and the combined target theory of a finite source theory are

$$\begin{aligned} \mathsf{Gen}(e) &= E_e +\!\!+ X_e, \\ \mathsf{Theory}(\varphi) &= \mathsf{Gen}(\varphi) +\!\!+ [t_\varphi], \\ \mathsf{Theory}^*(\Phi) &= \operatorname*{concat}_{\varphi\in\Phi} \mathsf{Theory}(\varphi). \end{aligned}$$

Lean keeps the lists separate so each formula's role remains explicit, then concatenates them in this fixed order for semantic statements. Lean calls the per-term record `TermTranslation` *and the pair of generated formula lists* `AuxiliaryTheory`*. The aggregate operation is Lean's* `translatedTheories`*; every sentence is translated against the same reified signature.* [24]

5.4 How terms are translated

Boolean constructors, ground variables, ground constants, equality at ground types, and quantifiers translate structurally. Three cases require additional machinery.

Complete applications. The translator collects the left-associated source application spine. When its result is ground, the typed argument indices prove that all leading arrow arguments are present, so it emits exactly one flattened source or application symbol.

Residual function values. A variable, constant, or partial application of arrow type cannot be emitted as an under-applied first-order symbol. It is eta-expanded into a closure. The `LambdaBody` structure opens the complete leading lambda telescope and the theorem `denote_etaBody` proves that the resulting source function has exactly the original denotation. [25]

Example 5.5 (A partial application becomes a closure). Let A, B, and C be base types, let $h : A \to B \to C$ be a source constant, and let $a : A$ be local. The complete application $h\,a\,b$ may be flattened directly to one first-order application $h_{\mathsf{flat}}(a, b) : C$. The partial application $h\,a$ still

[24] Lean formalization: `Crush/Metatheory/Defunctionalization/TermTranslation.lean` and `Defunctionalization/Flattened/Theory.lean`.

[25] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Lambda.lean` and `Defunctionalization/EtaCorrectness.lean`.

has type $B \to C$, so emitting $h_{\mathsf{flat}}(a)$ would create an ill-formed under-application. Instead the translator eta-expands it to $\lambda b{:}B.\, h\, a\, b$ and produces a closure symbol

$$\mathsf{clos}_{h,a} : \lfloor A \rfloor \longrightarrow \mathsf{Fn}(B, C)$$

with the defining equation

$$\forall a{:}\lfloor A \rfloor.\ \forall b{:}\lfloor B \rfloor.\quad \mathsf{app}_{B,C}(\mathsf{clos}_{h,a}(a), b) = h_{\mathsf{flat}}(a, b).$$

The closure captures exactly a; the global constant h remains a symbol and is not duplicated as a capture. This small case is the same mechanism used for the captured lambdas in the running example below.

Function equality. Equality between opaque function-sort values would be stronger than pointwise equality unless extensionality is asserted. For each function equality used by the translation, X_e receives a closed formula saying that pointwise equality over the complete arrow telescope implies equality of the function values.

Definition 5.6 (Flattened closure equation)**.** *Suppose χ has arrow type $\alpha = \tau_1 \to \cdots \to \tau_n \to \upsilon$ with ground υ, exact captures $\bar{y}$, and eta-opened body b. Its defining equation has the shape*

$$\forall \bar{y}\, x_1 \cdots x_n.\quad \mathsf{app}_\alpha(\mathsf{clos}_\chi(\bar{y}), x_1, \ldots, x_n) = t_b,$$

where app_α abbreviates the application symbol selected by the full arrow type α. The left side uses one flattened application, not a chain of unary applications. The right side carries all declarations and formulas recursively generated while translating b.

The three mutually recursive modes—ordinary term translation, application spine translation, and lambda-body translation—use a lexicographic measure built from constructor count and a mode tag. Lean therefore accepts them as total definitions, with no `partial` escape hatch. [26]

5.5 Running example: the complete flattened result

Example 5.7 (Step-by-step defunctionalization of (RE)). For the running arrow type,

$$\mathsf{args}(\mathsf{F}) = [\mathsf{T}], \qquad \mathsf{res}(\mathsf{F}) = \mathsf{T}, \qquad \lfloor \mathsf{F} \rfloor = \mathsf{Fn}(\mathsf{T}, \mathsf{T}).$$

Continue to abbreviate the last sort by F. Flattening introduces the fully applied symbol

$$\mathsf{app}_{\mathsf{T},\mathsf{T}} : \mathsf{F} \times \mathsf{T} \longrightarrow \mathsf{T}. \tag{A}$$

Thus neither $f\, x$ nor $f\, (f\, x)$ remains a first-order application node:

$$f\, x \quad \mapsto \quad \mathsf{app}(f, x), \qquad f\, (f\, x) \quad \mapsto \quad \mathsf{app}(f, \mathsf{app}(f, x)),$$

where this example suppresses the type subscripts on app.

The two lambda occurrences in (RE) produce distinct closure descriptors χ_L and χ_R. The left occurrence captures exactly f and the right occurrence captures exactly g, so their declarations are

$$\mathsf{clos}_{\chi_L} : \mathsf{F} \longrightarrow \mathsf{F}, \qquad \mathsf{clos}_{\chi_R} : \mathsf{F} \longrightarrow \mathsf{F}. \tag{C}$$

[26] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Translate.lean`.

The translated main sentence contains no lambda and no under-applied symbol:

$$t_{\mathsf{sq}} = \forall f{:}\mathsf{F}.\ \forall g{:}\mathsf{F}.\ \big(\big(\forall x{:}\mathsf{T}.\ \mathsf{app}(f,x) = \mathsf{app}(g,x)\big) \to \mathsf{clos}_{\chi_L}(f) = \mathsf{clos}_{\chi_R}(g)\big)\,. \tag{T}$$

The meaning of the closure values is supplied separately by the generated equations

$$\begin{aligned} E_L &= \forall f{:}\mathsf{F}.\ \forall x{:}\mathsf{T}.\ \mathsf{app}(\mathsf{clos}_{\chi_L}(f), x) = \mathsf{app}(f, \mathsf{app}(f,x)),\\ E_R &= \forall g{:}\mathsf{F}.\ \forall x{:}\mathsf{T}.\ \mathsf{app}(\mathsf{clos}_{\chi_R}(g), x) = \mathsf{app}(g, \mathsf{app}(g,x)). \end{aligned} \tag{E}$$

Finally, because (T) compares values of sort F, the generated extensionality list contains

$$X_{\mathsf{F}} = \forall p, q{:}\mathsf{F}.\ \big(\big(\forall x{:}\mathsf{T}.\ \mathsf{app}(p,x) = \mathsf{app}(q,x)\big) \to p = q\big)\,. \tag{X}$$

Up to repeated declaration occurrences, the complete result is therefore

$$\mathcal{F}[\![\varphi_{\mathsf{sq}}]\!] = \big(t_{\mathsf{sq}}, D_{\mathsf{sq}}, [E_L, E_R], [X_{\mathsf{F}}]\big),$$

where the distinct identities mentioned by D_{sq} are the application symbol (A) and the two closure symbols (C). The target theory for this one sentence is

$$\mathsf{Theory}(\varphi_{\mathsf{sq}}) = [E_L, E_R, X_{\mathsf{F}}, t_{\mathsf{sq}}]. \tag{R}$$

The recursive translator reaches (R) in four conceptually separate steps:

1. `translateSpineWith` sees f as the head of $f\,x$, translates the argument x, and finishes the spine only because its result is the ground sort T.
2. Repeating that operation for the outer application produces $\mathsf{app}(f, \mathsf{app}(f,x))$; no unary application chain is retained.
3. `translateLambdaBodyWith` opens the binder x, translates that ground body, and closes E_L or E_R over both the exact capture and the opened binder.
4. Ordinary `translateWith` replaces each residual function-valued lambda by its closure symbol, combines all recursive output, and adds X_{F} at the function-equality node.

This separation explains why the translated term, closure equations, and extensionality formula are different fields of `TermTranslation`: all are needed, but they are produced for different reasons.

6 The canonical model and flattened correctness

Contents of this section. Why the model is chosen; term preservation; generated-formula validity; unsatisfiability reflection.

6.1 Modeling function values

An arbitrary first-order target model may interpret $\mathsf{Fn}(\tau,\sigma)$ by any nonempty carrier. To prove soundness, however, it suffices to construct one target model from each source model. This is the role of the canonical model.

Definition 6.1 (Canonical flattened model)**.** *Given a source model M_s, its canonical target model $\widehat{M}_s$ uses the same carriers at Boolean and base sorts and uses the genuine source function space at each function sort:*

$$[\![\mathsf{Fn}(\tau,\sigma)]\!]_{\widehat{M}_s} = [\![\tau \to \sigma]\!]_{M_s}.$$

More generally, the Lean development exposes mutually inverse, type-directed maps $\mathsf{toCanonical}_\tau$ *and* $\mathsf{fromCanonical}_\tau$ *so statements remain uniform at all types. In $\widehat{M}_s$:*

- *flattened source symbols perform repeated source application;*
- $\mathsf{app}_{\tau,\sigma}$ *performs repeated source application;*
- clos_χ *reconstructs the exact captured source environment and denotes the source lambda.*

This does not claim that all target models contain actual functions. It is a countermodel construction used only in the soundness proof. [27]

Example 6.2 (Why every generated formula in (R) is true). Let V_{Tree} be the source carrier chosen for T. The canonical target carrier for F is the actual function space $V_{\mathsf{Tree}} \to V_{\mathsf{Tree}}$. It interprets

$$\begin{aligned}\mathsf{app}(h,x) &= h(x),\\ \mathsf{clos}_{\chi_L}(f) &= \lambda x.\, f(f(x)),\\ \mathsf{clos}_{\chi_R}(g) &= \lambda x.\, g(g(x)).\end{aligned}$$

Consequently E_L and E_R reduce to ordinary beta equations, and X_{F} is ordinary function extensionality. Under this interpretation, the first-order sentence t_{sq} has exactly the denotation of the higher-order sentence φ_{sq}. This is the concrete instance of the two general theorems below: term preservation handles t_{sq}, while generated-formula validity handles E_L, E_R, and X_{F}.

A source valuation ρ induces a canonical target valuation $\widehat{\rho}$ by applying $\mathsf{toCanonical}$ pointwise.

Theorem 6.3 (Flattened term preservation)**.** *For every source model M_s, valuation ρ, and typed source term $e : \mathsf{Term}_\Sigma(\Gamma,\tau)$:*[28]

$$[\![t_e]\!]_{\widehat{M}_s,\widehat{\rho}} = \mathsf{toCanonical}_\tau\big([\![e]\!]_{M_s,\rho}\big), \qquad \text{where } \mathcal{F}[\![e]\!] = (t_e, D_e, E_e, X_e).$$

Proof architecture. One structural induction proves ordinary translation and spine translation simultaneously. Ground syntax is homomorphic. Complete spines use the currying theorem for the flattened symbol. Residual arrows use eta-closure correctness. Quantifiers use the fact that extending a canonical target valuation corresponds to extending the source valuation. □

Theorem 6.4 (Validity of generated formulas)**.** *For every e, the same canonical model satisfies the entire auxiliary theory:*[29]

$$\widehat{M}_s \models^{\mathsf{T}} \mathsf{Gen}(e).$$

Proof architecture. A second simultaneous induction follows the three translator modes. Closure equations use term preservation and eta correctness; function extensionality is valid because the canonical function carriers are genuine source function spaces. □

[27] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean` and `Defunctionalization/Flattened/Symbol.lean`.

[28] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Denotation.lean`.

[29] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.

Theorem 6.5 (Flattened model extension)**.** *For every source sentence φ and source model M_s:*[30]

$$M_s \models \varphi \quad \Longrightarrow \quad \widehat{M_s} \models^{\mathsf{T}} \mathsf{Theory}(\varphi).$$

Proof. Generated-formula validity proves the prefix $\mathsf{Gen}(\varphi)$. Flattened term preservation at the empty valuation proves the final assertion t_φ. □

Theorem 6.6 (Aggregate flattened model extension)**.** *For every finite source theory Φ and source model M_s,*

$$M_s \models^{\mathsf{T}} \Phi \quad \Longrightarrow \quad \widehat{M_s} \models^{\mathsf{T}} \mathsf{Theory}^*(\Phi).$$

All source sentences use one canonical target model and one shared flattened symbol family; the theorem is not a pointwise collection of unrelated target models. [31]

Proof. Induction over Φ applies the singleton model-extension theorem at the head and the induction hypothesis to the tail. Target-theory satisfaction distributes over concatenation. □

Corollary 6.7 (Flattened unsatisfiability reflection)**.** *The singleton model-extension result yields:*[32]

$$\mathsf{Unsat}_{\mathsf{FO}}(\mathsf{Theory}(\varphi)) \quad \Longrightarrow \quad \mathsf{Unsat}_{\mathsf{HO}}(\varphi).$$

Proof. If φ had a source model, model extension would construct a target model of $\mathsf{Theory}(\varphi)$, contradicting the premise. □

Corollary 6.8 (Aggregate flattened unsatisfiability reflection)**.**

$$\mathsf{Unsat}_{\mathsf{FO}}(\mathsf{Theory}^*(\Phi)) \quad \Longrightarrow \quad \mathsf{Unsat}_{\mathsf{HO}}(\Phi).$$

This is the theorem `target_theories_unsat_implies_source_unsat`. [33]

6.2 The unary reference proof

The development retains a smaller unary reference translation. We write $\mathcal{D}[\![e]\!]$ for its result, which uses a binary application symbol at each arrow. A source/target logical relation is defined at Boolean, base, and arrow types; its arrow clause says that related function values produce related results when observed through unary application. The fundamental lemma proves

$$[\![e]\!]_{M_s,\rho_s} \approx_\tau [\![\mathcal{D}[\![e]\!]]\!]_{M_t,\rho_t}$$

under related models and valuations. The flattened currying development then proves that the formal flattened translation and the unary reference have the same canonical denotation. Thus the unary proof is an independent semantic reference, not the endpoint of the current VCG theorem.[34]

[30] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[31] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[32] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[33] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[34] Lean formalization: `Crush/Metatheory/Defunctionalization/Fundamental.lean`, `Defunctionalization/ModelExtension.lean`, and `Defunctionalization/Flattened/Currying.lean`.

7 Raw SMT syntax, typing, and semantics

Contents of this section. A declaration-aware type checker; relational term evaluation; standard theory interpretations; SMT datatype laws; semantic command-set equality.

The SMT syntax emitted by the Crush translator is intentionally not intrinsically typed. From this point onward, *raw* means that a syntax node does not carry a proof of its sort, in contrast to the typed HO and FO syntax above. Before raw commands enter the semantic soundness theorem, an executable, declaration-aware type system checks them. This keeps the syntax shared with the implementation without allowing an ill-sorted script to become "unsatisfiable" merely because no evaluation rule applies.

Definition 7.1 (Raw SMT model and evaluation)**.** *A raw model M_q contains one universe of values, a membership predicate* $\mathsf{inSort}(s, v)$*, a nonemptiness witness for every concrete sort, distinguished Boolean values, literal interpretations, and a graph* $\mathsf{apply}(p, \bar{v}, w)$ *for identifiers. The relation*

$$\mathsf{Eval}(M_q, \eta, q, v)$$

evaluates raw term q to v under nearest-binder-first environment η. It covers bound variables, literals, logical connectives, equality, quantifiers, annotations, and user-symbol applications. A closed raw formula q holds when it evaluates to semantic truth; only now do we write $M_q \models_{\mathsf{SMT}} q$.

The solver-facing predicate $\mathsf{Standard}(M_q)$ *additionally requires: the Boolean sort contains exactly the two distinguished Boolean values; the integer sort is in bijection with $\mathbb{Z}$; numeral literals and* `>=` *have their usual integer meanings; and every identifier graph is single-valued. These are semantic laws, not typing rules. In particular, a type system alone cannot prevent the literals* 0 *and* 1 *from collapsing in an arbitrary raw model. The Lean development includes a concrete standard-model witness, so this class of models is not empty.* [35]

Definition 7.2 (Declaration-aware SMT type checking)**.** *Let $\Sigma_q = (\Sigma_{\mathsf{sort}}, \Sigma_{\mathsf{fun}})$ record the arity of each preceding sort declaration and the argument/result sorts of each preceding symbol declaration. Let Γ_q record named local binders and Δ_q record the sorts of de Bruijn variables. The partial function*

$$\mathsf{infer}(\Sigma_q, \Gamma_q, \Delta_q, q) = \tau$$

computes the sort of q or reports an error. It checks literal and variable sorts, declared applications, the modeled logical and integer operators, quantifier bodies, simultaneous `let` *bindings, and annotations. The modeled fragment rejects unknown symbols and sorts, use before declaration, lambda terms, and theory constructs for which this document has no semantics.*

The command checker $\mathsf{check}(C)$ *processes the array from left to right. An assertion must infer as* `Bool`*. A nonrecursive* `define-fun` *body is checked before its new symbol enters Σ_q. For a* `define-funs-rec` *group, all signatures enter Σ_q together before any body is checked. For mutually recursive datatypes, all datatype sorts enter scope before field sorts are checked, followed by constructors and selectors. An indexed tester such as* `(_ is leaf)` *is accepted only when its index names a constructor introduced by a datatype declaration; an ordinary function returning the same sort is not a constructor. Duplicate declarations and empty recursive-definition groups are rejected. We define*

$$\mathsf{CommandsWellTyped}(C) \iff \mathsf{check}(C) = \mathsf{ok}.$$

This is an extrinsic type system over the existing command syntax, not a second typed SMT abstract syntax tree.

[35] Lean formalization: `Crush/Metatheory/SMT/Semantics.lean`.

The SMT subset covered by the current soundness theorem is deliberately small: Boolean literals and connectives, equality, integer numerals, and integer `>=`*. Quantifiers,* `let`*, declared uninterpreted functions, and the operations introduced by datatype declarations proved to satisfy the modeled datatype laws are also covered. Integer addition, subtraction, multiplication, division, and the bit-vector, array, and string theories are present in Crush's general SMT syntax but are rejected by* check *in metatheory mode. Consequently, the theorem below does not yet apply to an emitted command array containing those constructs.* [36]

Definition 7.3 (Raw datatype validity)**.** *Let B_q be one raw monomorphic* `declare-datatypes` *block. Its* structural check *requires that the block is nonempty, sort and operation identifiers are distinct, every sort has arity zero, type-parameter lists are empty, and every datatype has at least one constructor.*

The same block must also pass a finite-value check. For a datatype name d and fuel k, let $\mathsf{finite}(B_q, k, d)$ *hold when some constructor of d has only the following fields:*

- *an external complete sort that contains no sort declared by B_q; or*
- *a direct reference to a datatype d' in B_q for which* $\mathsf{finite}(B_q, k-1, d')$ *holds.*

The zero-fuel case is false. The executable productivity check requires $\mathsf{finite}(B_q, |B_q|, d)$ *for every declared d. The size bound is enough: if a finite dependency witness exists, a shortest one does not repeat a datatype name. A current-block occurrence hidden inside another sort constructor, such as* `(Array Int Tree)`*, is rejected because the modeled fragment has no positivity or nonemptiness proof for that enclosing theory.*

We write

$$\mathsf{DatatypesWellFormed}(B_q) \iff \mathsf{structural}(B_q) \wedge \mathsf{productive}(B_q).$$

Both conjuncts are executable Boolean checks shared literally by the modeled script validator, the canonical datatype-command certificate, and `Command.Supported`*. Thus a failed datatype condition rejects the command statically; it is not encoded by making command satisfaction false. For every non-datatype command,* `Command.Supported` *is true, while the ordinary modeled-fragment restrictions remain in* `CommandsWellTyped`*.* [37]

Example 7.4 (Productive and nonproductive declarations). The running tree is accepted because `leaf` is already a finite value; `node` may then refer directly to `Tree`:

```
(declare-datatypes ((Tree 0))
  (((leaf)
    (node (left Tree) (right Tree)))))
```

The declaration

```
(declare-datatypes ((Loop 0))
  (((mkLoop (next Loop)))))
```

is rejected. With one datatype, the only attempted constructor asks for a smaller-fuel `Loop` value and reaches the false zero-fuel case. Likewise, a mutual cycle with no base constructor is rejected, whereas a mutual tree/list block is accepted when `leaf` and `nil` provide finite base values. These cases are executable checker regressions. [38]

[36] Lean implementation: `Crush/SMT/Check.lean`.

[37] Lean implementations: `Crush/SMT/Check.lean`, `Crush/Metatheory/SMT/Semantics.lean`, and `Crush/Metatheory/SMT/DatatypeWF.lean`.

[38] Machine-checked examples: `Test/MetatheoryDatatype.lean`.

Definition 7.5 (Raw SMT datatype semantics)**.** *For one monomorphic* `declare-datatypes` *command, the predicate* $\mathsf{DatatypesHold}(M_q, B_q)$ *requires:*

1. *total, typed, injective constructors and matching selector equations;*
2. *total testers that accept their constructor and reject distinct ones;*
3. *disjoint constructor images and constructor exhaustiveness; and*
4. *a rank* $r : M_q.\mathsf{Value} \to \mathbb{N}$ *that strictly decreases on every field whose sort belongs to the same mutual block.*

The rank excludes cyclic or infinite raw values. Selectors are unconstrained on nonmatching constructors, exactly as in SMT-LIB.

Definition 7.6 (Raw function definitions)**.** *After the command checker has validated the relevant scope and signatures, each function graph is total at its declared type, and for typed arguments* $\bar{v}$*,*

$$\mathsf{apply}(f, \bar{v}, w) \quad \Longleftrightarrow \quad \mathsf{Eval}(M_q, \mathsf{reverse}(\bar{v}), q_f, w),$$

where q_f *is the retained body. Every body is evaluated against the same global graph. This gives* `define-funs-rec` *simultaneous mutual-recursion semantics. A nonrecursive* `define-fun` *reuses the same single-definition equation; its nonrecursive scope follows from checking the body before adding* f *to the declaration environment.*

Definition 7.7 (Command satisfaction and command-set equality)**.** *The judgment* $M_q \models_{\mathsf{SMT}} C$ *means that one raw model satisfies every command occurring in array* C*. It is deliberately insensitive to order and repeated occurrences. Define*

$$C_1 \equiv_{\mathsf{cmd}} C_2 \quad \Longleftrightarrow \quad (\forall c \in C_1.\ c \in C_2) \wedge (\forall c \in C_2.\ c \in C_1).$$

Then $C_1 \equiv_{\mathsf{cmd}} C_2$ *implies* $M_q \models_{\mathsf{SMT}} C_1 \leftrightarrow M_q \models_{\mathsf{SMT}} C_2$*. Concrete declaration-before-use ordering is checked separately by the Crush script validator.*

Definition 7.8 (Semantic command unsatisfiability)**.**

$$\begin{aligned} \mathsf{Unsat}_{\mathsf{SMT}}(C) \quad \Longleftrightarrow \quad & \mathsf{CommandsSupported}(C) \\ & \wedge\, \mathsf{CommandsWellTyped}(C) \\ & \wedge\, \forall M_q.\ \mathsf{Standard}(M_q) \Rightarrow \neg(M_q \models_{\mathsf{SMT}} C). \end{aligned}$$

The first conjunct supplies the additional free-datatype conditions. The second rejects malformed or semantically unmodeled syntax instead of allowing a missing evaluation to prove unsatisfiability. The last conjunct uses the standard meanings of the modeled SMT theories. It is a mathematical premise about commands, not by itself a theorem connecting an external solver process to this predicate. [39]

8 Exact FO-to-SMT representation

Contents of this section. One shared encoding; semantic command-set representation; SMT datatype representation proofs; model lifting.

[39] Lean formalization: `Crush/Metatheory/SMT/Semantics.lean`.

Definition 8.1 (Encoding choice)**.** *For a typed symbol family S, an encoding $\mathcal{E}$ chooses:*

- *an injective concrete SMT sort $\mathcal{E}_s(\kappa)$ for every FO sort, with* Bool *mapped to SMT* `Bool`*;*
- *an identifier $\mathcal{E}_i(p)$ for every typed symbol, injective both across declaration indices and within each symbol-family fiber, and fresh from logical built-ins;*
- *predicates identifying sorts and symbols declared by SMT datatype commands;*
- *the exact SMT datatype command prefix C_{data}; and*
- *ordinary printable names for every other symbol.*

All components share this one namespace and one raw model. SMT datatypes do not introduce a parallel term encoder. Given $\mathcal{E}$, we now write

$$\mathcal{A}[\![t]\!][\mathcal{E}]$$

for the raw encoding of typed FO term t. Variables become numeric de Bruijn indices; symbols use $\mathcal{E}_i$; and logical syntax is mapped homomorphically. [40]

Definition 8.2 (Pure theory encoder)**.** *For an ordered declaration list D and typed theory T, define*

$$\mathsf{theory}(\mathcal{E}, D, T) = C_{\mathsf{data}} +!+ C_{\mathsf{sort}} +!+ C_{\mathsf{decl}} +!+ C_{\mathsf{assert}}.$$

SMT datatype sorts and symbols are omitted from the ordinary declaration segments. Used ordinary sorts, declarations, and assertions retain stable source order. [41]

Definition 8.3 (Exact semantic theory representation)**.** *Only after defining the pure encoder and command-set equality do we introduce*

$$\mathsf{Rep}(\mathcal{E}, T, C) \quad \Longleftrightarrow \quad \exists D.\ C \equiv_{\mathsf{cmd}} \mathsf{theory}(\mathcal{E}, D, T).$$

This is `TheoryRepresentation`*. It is intentionally weaker than array equality and exactly as strong as the raw command semantics: the Crush translator may interleave independent declarations and assertions or suppress duplicates, but it may neither omit nor add a semantically relevant command.* [42]

Theorem 8.4 (Exact aggregate representation)**.** *Let D_Φ concatenate every declaration occurrence produced while translating the finite source theory Φ. The aggregate encoder is*

$$\mathsf{Commands}(\mathcal{E}, \Phi) = \mathsf{theory}(\mathcal{E}, D_\Phi, \mathsf{Theory}^*(\Phi)),$$

and

$$\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}^*(\Phi), \mathsf{Commands}(\mathcal{E}, \Phi)).$$

The theorem is `encode_theories`*;* `encode_translation` *is its singleton counterpart.* [43]

[40] Lean formalization: `Crush/Metatheory/SMT/Representation.lean`.
[41] Lean formalization: `Crush/Metatheory/SMT/Representation.lean`.
[42] Lean formalization: `Crush/Metatheory/SMT/Representation.lean`.
[43] Lean formalization: `Crush/Metatheory/SMT/Representation.lean`.

Example 8.5 (Concrete names for the flattened running example). One injective encoding may choose

$$\begin{aligned} \mathsf{T} &\mapsto \texttt{Tree}, \\ \mathsf{F} &\mapsto \texttt{Fn\_Tree\_Tree}, \\ \mathsf{app}_{\mathsf{T},\mathsf{T}} &\mapsto \texttt{app\_Tree}, \\ \mathsf{clos}_{\chi_L} &\mapsto \texttt{clos\_left}, \\ \mathsf{clos}_{\chi_R} &\mapsto \texttt{clos\_right}. \end{aligned}$$

The function-side declaration segment then contains commands of the following shape:

```
(declare-sort Fn_Tree_Tree 0)
(declare-fun app_Tree
  (Fn_Tree_Tree Tree) Tree)
(declare-fun clos_left
  (Fn_Tree_Tree) Fn_Tree_Tree)
(declare-fun clos_right
  (Fn_Tree_Tree) Fn_Tree_Tree)
```

The encoded assertions are the untyped images of $E_L, E_R, X_{\mathsf{F}}, t_{\mathsf{sq}}$ from (R). This example separates two questions that are easy to conflate: the flattened pass determines the typed symbols and formulas, while $\mathcal{E}$ only chooses concrete, globally fresh SMT names for them.

8.1 SMT datatype representation

Definition 8.6 (One represented SMT datatype block)**.** *For a source block B, symbol map S, shared encoding $\mathcal{E}$, and allocated raw block encoding A, write $A_s(d)$ for the concrete SMT sort name allocated to member d. A* $\mathsf{Representation}(B, S, \mathcal{E}, A)$ *proves:*

- *the exact emitted* `declare-datatypes` *command is well formed;*
- *each constructor, selector, and tester has exactly one datatype role;*
- *every datatype sort and symbol is marked as declared by the datatype command; and*
- *all encoded sorts and identifiers equal the names in A, including indexed tester identifiers.*

An environment representation lists such proofs in dependency order and proves that their command sequence is exactly C_{data}. [44]

Definition 8.7 (One custom datatype across HO, FO, and SMT)**.** *For an accepted ground Lean datatype instance, structural reification chooses a block member d and its identity $\beta_{B,d}$. The sort component of the lowering pipeline is*

$$\boxed{\textit{ground Lean datatype instance}} \xrightarrow{\textit{structural reification}} \boxed{\mathsf{Base}(\beta_{B,d})\ \textit{in HO}}$$

$$\boxed{\mathsf{Base}(\beta_{B,d})\ \textit{in HO}} \xrightarrow{\mathsf{erase}} \boxed{\mathsf{Base}(\beta_{B,d})\ \textit{in FO}} \xrightarrow{\mathcal{E}_s} \boxed{A_s(d)\ \textit{in raw SMT}}$$

The first arrow records declaration structure and typed block membership; it is not a denotational semantics for arbitrary `Lean.Expr`*. The semantic correspondence begins with* $\mathsf{FreeData}_B(S, M_s)$*, which identifies the HO base carrier with the canonical finite value family* $\mathsf{Val}_B(M_s.B, d)$*.*

[44] Lean formalization: `Crush/Metatheory/SMT/DatatypeWF.lean and SMT/DatatypeRepresentation.lean`.

The operation component follows the same symbol map. A selected HO constructor, selector, or tester becomes its declaration-indexed FO source symbol, and $\mathcal{E}_i$ maps that symbol to the corresponding constructor, selector, or indexed tester identifier recorded by A. The SMT `declare-datatypes` *command declares those identifiers together, so the same object is not also emitted as an ordinary uninterpreted sort or function. All other HO and FO syntax remains generic: datatype-specific meaning enters only through the block symbol map, the free-datatype source-model condition, and the SMT datatype representation proof.* [45]

Example 8.8 (The SMT tree declaration). For the later guarded specialization in which source Num values are represented by SMT integers, the allocated datatype command may be printed as follows (irrelevant presentation details omitted):

```
(declare-datatypes ((Tree 0))
  (((leaf (leaf_value Int))
    (node (left Tree) (right Tree)))))
```

The command simultaneously introduces the sort, constructors, selectors, and implicit testers such as `(_ is leaf)`. Its SMT datatype laws say that every raw `Tree` value is a finite constructor tree. They do *not* yet say that the integer stored by `leaf` represents a source Num value. That additional condition is exactly the guard introduced in Section 9.

Theorem 8.9 (SMT datatype command validity)**.** *If M_s satisfies the free-datatype model condition for the represented datatype environment $\mathcal{D}$, then the raw model induced by $\mathcal{E}$ and the canonical FO model satisfies the complete SMT datatype command prefix:*

$$\mathsf{FreeData}_{\mathcal{D}}(M_s) \quad \Longrightarrow \quad M_q(\mathcal{E}, \widehat{M_s}) \models_{\mathsf{SMT}} C_{\mathsf{data}}.$$

The proof establishes constructor typing and injectivity, selector/tester laws, exhaustiveness, disjointness, and rank decrease in the shared raw universe. The dependency-extension and transport modules prove that each earlier block remains valid after every later disjoint block is installed. [46]

Theorem 8.10 (Non-vacuity of represented datatype commands)**.** *Assume the same free-datatype source model and an integer view giving the shared target the standard interpretation required by the modeled SMT fragment. Then there exists one* standard *raw SMT model satisfying the represented datatype declaration. For a dependency-ordered datatype environment, one standard model satisfies the complete native datatype prefix. Consequently neither the single declaration nor that prefix can satisfy* $\mathsf{Unsat}_{\mathsf{SMT}}$ *merely because datatype commands were installed.*

The proof reuses the canonical datatype model, the existing transport theorem for fresh graph extensions, and the standard integer model. It does not define a second datatype interpretation. [47]

8.2 One induced raw model

Definition 8.11 (Induced raw model)**.** *Given $\mathcal{E}$ and a typed target model M_t, the induced raw universe tags each value by its typed FO sort. Sort membership checks that tag. The graph for an encoded symbol decodes typed arguments, applies M_t's curried interpretation, and retags the result. An optional* `ExtraGraph` *adds derived identifiers together with a proof that they are disjoint from all identifiers selected by $\mathcal{E}$.* [48]

[45] Lean formalization: `Crush/Metatheory/Reification/Datatype.lean, Datatype/Flattened.lean, SMT/Datatype.lean, and SMT/DatatypeRepresentation.lean`.

[46] Lean formalization: `Crush/Metatheory/SMT/DatatypeCanonical.lean, SMT/DatatypeLifted.lean, and SMT/DatatypeTransport.lean`.

[47] Lean formalization: `Crush/Metatheory/SMT/DatatypeTransport.lean`.

[48] Lean formalization: `Crush/Metatheory/SMT/Model.lean and SMT/ModelExtension.lean`.

Theorem 8.12 (Complete representation soundness)**.** *Let $\mathcal{D}$ be the datatype environment represented by $\mathcal{E}$. If*

$$\mathsf{Rep}(\mathcal{E}, T, C), \qquad \mathsf{FreeData}_{\mathcal{D}}(M_s), \qquad \widehat{M_s} \models^{\mathsf{T}} T,$$

then there exists one raw model M_q with $M_q \models_{\mathsf{SMT}} C$. SMT datatype commands, ordinary sort declarations, ordinary symbol declarations, and all assertions are validated in that same model. [49]

Model construction. The term-evaluation theorem relates $\mathcal{A}[\![t]\!][\mathcal{E}]$ to typed denotation in M_t. Separate lemmas validate ordinary declarations and assertions. SMT datatype validity supplies the exact prefix. Finally, $C \equiv_{\mathsf{cmd}} \mathsf{theory}(\mathcal{E}, D, T)$ transports satisfaction to the represented emitted command sequence. □

Corollary 8.13 (Aggregate lowering reflection)**.** *If*

$$\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}^*(\Phi), C) \quad \textit{and} \quad \mathsf{Unsat}_{\mathsf{SMT}}(C),$$

then

$$\mathsf{Unsat}^{\mathcal{D}}_{\mathsf{HO}}(\Phi).$$

This is `commands_unsat_implies_source_theory_unsat`. It composes complete representation soundness with aggregate flattened model extension. [50]

9 Representing source values inside SMT types

Contents of this section. Subset representations; related FO models; recursive datatype guards; one guard-aware SMT-model theorem.

The Crush translator can represent a source type using an existing SMT type together with a predicate that selects exactly the values belonging to the source type. For example, it represents $\mathbb{N}$ using $\mathbb{Z}$ and the predicate $0 \le z$. We call the selecting predicate a *guard*. A custom datatype uses the SMT datatype emitted for it, and its guard recursively requires each field to satisfy the guard of that field's source type.

9.1 Subset representations

Definition 9.1 (Guarded-subset representation)**.** *A guarded-subset representation $R : S \rightsquigarrow T$ contains*

$$\mathsf{encode} : S \to T, \qquad \mathsf{guard} : T \to \mathsf{Prop}, \qquad \mathsf{decode} : (t : T) \to \mathsf{guard}(t) \to S,$$

together with nonemptiness of S and the laws

$$\mathsf{guard}(\mathsf{encode}(s)), \qquad \mathsf{decode}(\mathsf{encode}(s)) = s, \qquad \mathsf{guard}(t) \Rightarrow \mathsf{encode}(\mathsf{decode}(t)) = t.$$

Thus S is equivalent to the subtype of T selected by the guard. Encoding is injective; source equality is preserved and reflected; and source quantifiers correspond to the following guarded target quantifiers.[51]

$$\forall t : T.\ \mathsf{guard}(t) \Rightarrow P(t), \qquad \exists t : T.\ \mathsf{guard}(t) \wedge P(t).$$

[49] Lean formalization: `Crush/Metatheory/SMT/Soundness.lean`.
[50] Lean formalization: `Crush/Metatheory/SMT/Soundness.lean`.
[51] Lean formalization: `Crush/Metatheory/Guarded/Encoding.lean`.

Example 9.2 (The external Num field represented by integers). Specialize the source carrier of Num to $\mathbb{N}$ and the target carrier to $\mathbb{Z}$. The representation is

$$\mathsf{encode}(n) = \mathsf{ofNat}(n), \qquad \mathsf{guard}(z) \equiv 0 \le z, \qquad \mathsf{decode}(z, p) = \mathsf{toNat}(z).$$

The proof $p : 0 \le z$ is what makes encoding after decoding recover the original integer. For example, a source quantifier

$$\forall n{:}\mathsf{Num}.\ P(n)$$

is represented by

$$\forall z{:}\mathsf{Int}.\ 0 \le z \to P'(z),$$

whereas an existential uses $0 \le z \wedge P'(z)$. Negative integers remain in the SMT carrier but are excluded from the represented source values.

Definition 9.3 (FO carrier and model relation). *A carrier relation R assigns a guarded-subset representation to every nonlogical FO carrier; Boolean propositions use the identity relation. For source and target family models M_0, M_1, a model relation $M_0 \sim_R M_1$ requires every typed symbol interpretation to map encoded arguments to the encoded source result. This relational form admits both the generic pointwise lifting and SMT datatype interpretations.* [52]

Definition 9.4 (Guard-restricted FO denotation). *For target model M_1, relation R, and lifted valuation, guarded denotation uses ordinary semantics for variables, symbols, connectives, and equality, but restricts both quantifiers to the selected carrier guard. It is written explicitly as*

$$\mathsf{guardDenote}(e, M_1, R, \nu).$$

No bracket notation is introduced because ordinary denotation and guarded denotation must remain visually distinct. [53]

Theorem 9.5 (FO guarded preservation). *If $M_0 \sim_R M_1$, then for every typed term e and source valuation ρ,*

$$\mathsf{guardDenote}(e, M_1, R, \mathsf{lift}_R(\rho)) = R_\kappa.\mathsf{encode}([\![e]\!]_{M_0,\rho}),$$

where κ is the result sort of e. The theorem handles variables, symbols, all logical forms, equality, and both quantifiers uniformly. [54]

9.2 Datatype guard lifting

Definition 9.6 (Structural datatype guard). *Let external fields be related pointwise by R_0. A target datatype value is well formed when every external payload satisfies its base guard and every recursive payload is structurally well formed. This predicate is $\mathsf{WF}_B(R_0, v)$. Encoding maps the constructor tree recursively and encoding or decoding is inverse exactly on WF_B values. Hence every productive block lifts R_0 to a guarded relation on each of its datatype carriers.* [55]

Theorem 9.7 (Selector-form guard equivalence). *The Crush translator defines a predicate `wf_T` as a conjunction over every constructor tester and its selectors: when a tester accepts, every matching selected field must satisfy its guard. For a productive block,*

$$\mathsf{WF}_B(R_0, v) \iff \mathsf{SelWF}_B(R_0, v).$$

[52] Lean formalization: `Crush/Metatheory/FO/Guarded.lean`.
[53] Lean formalization: `Crush/Metatheory/FO/Guarded.lean`.
[54] Lean formalization: `Crush/Metatheory/FO/Guarded.lean`.
[55] Lean formalization: `Crush/Metatheory/Datatype/Guarded.lean`.

This is `Val.wf_iff_selWF`*. The enlarged carrier model installs SMT datatype constructors, selectors, and testers while preserving the same per-symbol model relation.* [56]

Example 9.8 (The recursive guard for the running tree). Lifting the $\mathbb{N} \rightsquigarrow \mathbb{Z}$ representation through the tree block gives

$$\begin{aligned} \mathsf{WF}_{\mathsf{Tree}}(\mathsf{leaf}(z)) &\iff 0 \le z, \\ \mathsf{WF}_{\mathsf{Tree}}(\mathsf{node}(l, r)) &\iff \mathsf{WF}_{\mathsf{Tree}}(l) \wedge \mathsf{WF}_{\mathsf{Tree}}(r). \end{aligned} \tag{G}$$

The selector-form predicate expresses the same condition without pattern matching:

$$\begin{aligned} \mathsf{wf}_{\mathsf{Tree}}(t) \iff &\big(\mathsf{isLeaf}(t) \to 0 \le \mathsf{leafValue}(t)\big) \\ &\wedge \big(\mathsf{isNode}(t) \to \mathsf{wf}_{\mathsf{Tree}}(\mathsf{left}(t)) \wedge \mathsf{wf}_{\mathsf{Tree}}(\mathsf{right}(t))\big). \end{aligned} \tag{PG}$$

SMT constructor exhaustiveness and tester disjointness make (G) and (PG) equivalent. This is the concrete content of `Val.wf_iff_selWF`; it is why an SMT datatype carrier can be larger than the source datatype carrier while still representing it exactly.

9.3 Guard-aware raw syntax

Definition 9.9 (Guarded encoding)**.** *A guarded encoding $\mathcal{G} = (\mathcal{E}, g)$ reuses one ordinary encoding $\mathcal{E}$ and optionally supplies a raw guard term $g(\kappa, q)$ for a raw term q of sort κ. We now write*

$$\mathcal{G}[\![e]\!][\mathcal{G}]$$

for the shared term encoder instantiated with g. At a universal binder it emits guard implication; at an existential binder it emits guard conjunction. Every other term constructor is encoded by the same function used for $\mathcal{A}[\![e]\!][\mathcal{E}]$. The Lean structure carrying $\mathcal{E}$ and g is `SMT.GuardedEncoding`*.* [57]

Example 9.10 (Guarding the flattened running formula). Under the running representation, a tree binder in (T), (E), or (X) is encoded with $\mathsf{wf}_{\mathsf{Tree}}$ as its restriction. For example, the pointwise premise inside t_{sq} becomes schematically

$$\forall x{:}\mathtt{Tree}.\ \mathsf{wf}_{\mathsf{Tree}}(x) \to \mathtt{app\_Tree}(f, x) = \mathtt{app\_Tree}(g, x).$$

The chosen relation treats the function-value sort F as total, so its binders need no nontrivial guard. All uses of `app_Tree`, `clos_left`, and `clos_right` still come from the same ordinary encoding $\mathcal{E}$; guarding changes quantifier domains, not symbol identity.

Definition 9.11 (Guarded theory representation)**.** *Let C_{derived} be a specified command segment whose validity is proved separately, such as the mutual* `wf_T` *definitions. Define the following guarded command sequence and representation.*[58]

$$\mathsf{gtheory}(\mathcal{G}, C_{\mathsf{derived}}, D, T) = C_{\mathsf{data}} +!+ C_{\mathsf{derived}} +!+ C_{\mathsf{ordinary}} +!+ C_{\mathsf{gassert}}.$$

Then

$$\mathsf{Rep}_g(\mathcal{G}, C_{\mathsf{derived}}, T, C) \iff \exists D.\ C \equiv_{\mathsf{cmd}} \mathsf{gtheory}(\mathcal{G}, C_{\mathsf{derived}}, D, T).$$

Definition 9.12 (Semantic guard contract)**.** *For the induced raw model, target model M_1, and carrier relation R, the contract* $\mathsf{GuardSem}(\mathcal{G}, M_1, R)$ *says:*

[56] Lean formalization: `Crush/Metatheory/Datatype/Guarded.lean`, `Datatype/Carrier.lean`, `Datatype/FamilyModel.lean`, and `Datatype/Flattened.lean`.

[57] Lean formalization: `Crush/Metatheory/SMT/Guarded.lean`.

[58] Lean formalization: `Crush/Metatheory/SMT/Guarded.lean`.

- *if syntax is omitted for κ, every target value satisfies $R_\kappa.\mathsf{guard}$; and*
- *if $g(\kappa, q)$ is emitted and q evaluates to v, the guard term evaluates exactly to $R_\kappa.\mathsf{guard}(v)$.*

The compositional `TermSemantics` form applies the same statement to an arbitrary already-evaluated raw term, which is needed for datatype selectors. [59]

Theorem 9.13 (Guarded raw term preservation)**.** *If $M_0 \sim_R M_1$ and the guarded encoding has the semantic guard contract, then the raw term $\mathcal{G}[\![e]\!][\mathcal{G}]$ evaluates to the tagged encoding of $[\![e]\!]_{M_0,\rho}$. This is `guardTerm_rel_eval`; it composes FO guarded preservation with the raw evaluator.* [60]

9.4 SMT datatype blocks and recursive guards in one model

The target model is built one datatype block at a time in dependency order. At each step, the current block receives its free-datatype carrier and SMT datatype interpretations. A `DependencyOrdered` proof states that every earlier datatype command is disjoint from every later block, not only from its immediate successor. The following lemmas preserve constructor, selector, tester, exhaustiveness, and rank laws as each later block is added.

Each represented block has one exact `define-funs-rec` guard command. Its retained names and binder determine the same shared `wfBody` syntax used by the Crush translator. The raw evaluator proves the Boolean conjunction and field clauses, then `wfDefs_valid` proves the simultaneous command graph. The exact guard command sequence is indexed by the same represented block list as the SMT datatype prefix. [61]

For the running tree, the one-function recursive block has the following schematic SMT-LIB shape:

```
(define-funs-rec
  ((wf_Tree ((t Tree)) Bool))
  ((and
     (=> ((_ is leaf) t)
         (>= (leaf_value t) 0))
     (=> ((_ is node) t)
         (and (wf_Tree (left t))
              (wf_Tree (right t)))))))
```

This is a concrete command, not an axiom schema: `wfDefs_valid` proves that its simultaneous function graph denotes (PG) in the same raw model as the SMT `Tree` declaration.

Theorem 9.14 (Complete guarded model lifting)**.** *Assume*

$$\mathsf{Rep}_g(\mathcal{G}, C_{\mathsf{derived}}, T, C), \quad M_0 \sim_R M_1, \quad M_0 \models^{\mathsf{T}} T,$$

the semantic guard contract, validity of C_{data}, and validity of C_{derived}. Then one raw model satisfies C. This is `guarded_lift`; SMT datatype declarations, derived recursive definitions, ordinary declarations, and guarded assertions all share one raw graph. [62]

[59] Lean formalization: `Crush/Metatheory/SMT/GuardedSoundness.lean`.
[60] Lean formalization: `Crush/Metatheory/SMT/GuardedSoundness.lean`.
[61] Lean formalization: `Crush/Metatheory/SMT/Datatype.lean, SMT/DatatypeGuard.lean, SMT/DatatypeGuarded.lean, and SMT/DatatypeTransport.lean`.
[62] Lean formalization: `Crush/Metatheory/SMT/GuardedSoundness.lean`.

Remark 9.15 (Why the uniform guard interpretation matters). A model-indexed target construction may be useful as a low-level premise, but placing it inside the definition of source unsatisfiability would silently restrict the quantified source-model class. The uniform interpretation used at the translator boundary instead supplies a target construction for every source model satisfying the free-datatype condition. The resulting reflection theorem concludes $\mathsf{Unsat}_{\mathsf{HO}}^{\mathcal{D}}$ with no additional target-model assumption in the quantified source-model class. [63]

10 Lean reification and the proof-defined VCG

Contents of this section. Partial structural reification; common-environment theories; verified finite-theory command generation.

10.1 The supported Lean boundary

Definition 10.1 (Structural reification witness)**.** *The executable reifier recognizes supported normalized Lean-expression constructors and returns an existentially typed HO term. A witness retains the recognized expression shape, the reified term shape, their executable correspondence proof, and the exact* `ReifiedContext`*,* `ReifiedSignature`*, and* `DatatypeSignaturePrefix`*. Lean's inferred type is checked at every accepted node.* [64]

The witness is structural. It does not define a denotation for arbitrary Lean expressions and therefore does not by itself prove that the original Lean proposition and its reified sentence have equal truth values. Unsupported syntax returns no witness.

Definition 10.2 (Reified finite theory)**.** *Given the exact ordered expression list*

$$[E_1, \ldots, E_n],$$

common-environment reification first selects one datatype environment $\mathcal{D}$ and one ordinary signature tail. It then retains

$$(E_1, \varphi_1), \ldots, (E_n, \varphi_n)$$

pointwise in the same order, with every φ_i indexed by the same complete signature. The type `ReifiedSentencesFor` *prevents omission, reordering, or substitution of a fact. The partial executable function* `reifyTheory?` *returns this package only when every fact reifies.* [65]

Definition 10.3 (Supported datatype reification)**.** *Datatype discovery produces dependency-ordered, monomorphic, productive ground blocks. It rejects indices, dependent or function-valued fields, non-productivity, unsafe indirect recursion, recursors, and quotient primitives. Discovery is read-only; name allocation and command emission begin only after the complete structural plan is known.* [66]

Example 10.4 (A Lean expression corresponding to (RE)). The front end may encounter a normalized proposition with this source shape:

[63] Lean formalization: `Crush/Metatheory/VCG/Datatype.lean`.

[64] Lean formalization: `Crush/Metatheory/Reification/Type.lean, Reification/Term.lean, Reification/Capture.lean, Reification/Reify.lean, and Reification/Witness.lean`.

[65] Lean formalization: `Crush/Metatheory/Reification/Witness.lean`.

[66] Lean formalization: `Crush/Metatheory/Reification/Datatype.lean`.

```
inductive Tree where
  | leaf : Nat -> Tree
  | node : Tree -> Tree -> Tree

def runningFact : Prop :=
  forall f g : Tree -> Tree,
    (forall x : Tree, f x = g x) ->
    (fun x => f (f x)) = (fun x => g (g x))
```

Successful reification retains the ground datatype block for `Tree`, the external representation information for `Nat`, and the reified sentence φ_{sq}. It does not retain the printed binder names: typed de Bruijn references record the same binding structure. If any node falls outside the supported fragment, reification returns no witness rather than silently approximating the expression.

10.2 Pure aggregate VCG

Definition 10.5 (Pure VCG commands)**.** *Here VCG abbreviates* verification-condition generator*: it is the pure function that turns the reified finite theory into the command sequence whose unsatisfiability is checked. For encoding $\mathcal{E}$ and finite reified higher-order theory Φ, the pure VCG function computes*

$$\mathsf{theoryCommands}(\mathcal{E}, \Phi) = \mathsf{Commands}(\mathcal{E}, \Phi)$$

and retains

$$\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}^*(\Phi), \mathsf{theoryCommands}(\mathcal{E}, \Phi)).$$

For guarding policy $\mathcal{G}$ and exact derived segment C_{derived}, `guardedTheoryCommands` constructs the corresponding Rep_g witness. [67]

Example 10.6 (The verified command groups for the running fact). For the singleton theory $[\varphi_{\mathsf{sq}}]$, guarded generation combines the components already displayed:

C_{data}	the SMT `Tree` declaration,
C_{derived}	the recursive `wf_Tree` definition,
C_{ordinary}	the function sort and (A), (C) declarations,
C_{gassert}	guarded encodings of $E_L, E_R, X_{\mathsf{F}}, t_{\mathsf{sq}}$.

The VCG representation theorem does not merely say that this array was emitted: it proves command-set equivalence with the guarded encoder's output. The guarded model-lifting theorem then constructs one SMT model in which the encoded formulas from (R), the SMT datatype declaration, and the recursive guard definition are all satisfied together.

Theorem 10.7 (Finite-theory command-generation soundness)**.** *Let $C = \mathsf{theoryCommands}(\mathcal{E}, \Phi)$, let $\mathcal{D}$ be represented by the same encoding, and assume $\mathsf{Unsat}_{\mathsf{SMT}}(C)$. Then*

$$\mathsf{Unsat}^{\mathcal{D}}_{\mathsf{HO}}(\Phi).$$

This is `theoryCommands_unsat_implies_source_unsat`. The guarded representation above is composed with SMT datatype and guard validity by the command-equivalence theorem in the next section. [68]

[67] Lean formalization: `Crush/Metatheory/VCG/Generate.lean`.
[68] Lean formalization: `Crush/Metatheory/VCG/Generate.lean`.

11 Comparing emitted commands with the formal encoding

Contents of this section. Command locations; representation proofs; checked final-array comparison; reflection.

The proof-defined VCG constructs a canonical command sequence. The Crush translator may interleave declarations and assertions, omit duplicate occurrences, attach names to top-level assertions, and invoke extension handlers. Array equality would reject harmless reordering and duplicate removal, so the comparison instead proves that the two arrays contain the same commands under the membership-based semantics defined above.

11.1 Locating datatype commands in the emitted sequence

Definition 11.1 (Fact translation record)**.** *A* `FactTranslationRecord` *retains:*

- *the source expression, its datatype environment, its ordinary signature and constants, and the reified higher-order sentence when the expression is in the supported fragment;*
- *the complete command sequence* C_{emit} *emitted after all selected facts;*
- *the positions of its SMT datatype declarations in that sequence; and*
- *the positions of its recursive guard definitions in that sequence.*

Each recorded position comes with a Lean proof that the command at that position belongs to the corresponding reified datatype block. These positions are recomputed after all facts are translated, so every `FactTranslationRecord` *refers to the same emitted command sequence rather than an intermediate prefix.* [69]

Definition 11.2 (Representation evidence for one translated fact)**.** *For a fact translation record* P*:*

- $P.\mathsf{Representation}(\mathcal{E})$ *links every located SMT datatype command to one shared encoding and proves that the encoding's datatype prefix is exactly the retained command list;*
- $P.\mathsf{GuardDefinitionEncoding}(\mathcal{G})$ *links the recursive guard definitions to fresh, injective SMT identifiers independently of any model; and*
- $P.\mathsf{GuardDefinitionSemantics}(\mathcal{G})$ *realizes that static guard representation for every source model satisfying the free-datatype condition.*

These records connect emitted commands to the already-proved formal encoding. They do not define a second term encoder or a second datatype soundness theorem. [70]

11.2 Whole-theory comparison

Definition 11.3 (Semantically transparent normalization)**.** *Emitted assertions may have a root* `:named` *annotation, while the proof-defined encoder emits the unannotated body. A proved normalizer removes only that root annotation, and its theorem proves command satisfaction unchanged. A leading* `set-logic` *command is likewise proved semantically neutral. No other command is erased. The Lean normalizer is* `stripAssertionAnnotation`*.* [71]

[69] Lean formalization: `Crush/Metatheory/VCG/Datatype.lean`.
[70] Lean formalization: `Crush/Metatheory/VCG/Datatype.lean`.
[71] Lean formalization: `Crush/Metatheory/VCG/CommandEquivalence.lean`.

Definition 11.4 (Whole-theory agreement)**.** *Let W be an exact* ***`ReifiedSentencesFor`*** *witness with reified higher-order theory Φ_W. The existing* ***`SMT.GuardedTheoryRepresentation`*** *proposition is*

$$\mathsf{Rep}_g(\mathcal{G}, C_{\mathsf{guard}}, \mathsf{Theory}^*(\Phi_W), \mathsf{strip}(C_{\mathsf{emit}})).$$

The executable ***`CommandEquivalence.build?`*** *computes the expected guarded commands and checks inclusion in both directions. On success it returns this existing representation proposition directly; there is no second wrapper with the same mathematical content. Thus reordering and duplicate removal are accepted, while a missing or extra semantically relevant command is rejected.* [72]

Example 11.5 (What final-array agreement checks). Suppose the Crush translator emits the running commands in a different legal order, reuses one declaration instead of repeating its occurrence, and attaches the name `crush_fact_0` to the root of the encoded t_{sq} assertion. After removing that one root annotation, the comparison accepts the array if it contains the SMT `Tree` declaration, `wf_Tree`, every ordinary declaration, and the encoded formulas $E_L, E_R, X_{\mathsf{F}}, t_{\mathsf{sq}}$. It rejects, for example, an array that omits E_L or adds an unrelated assertion. This illustrates why the final theorem is about exact semantic command-set agreement rather than byte-for-byte script equality.

Theorem 11.6 (Emitted-command whole-theory reflection)**.** *Let P be a fact translation record with SMT datatype representation, static guard representation, uniform guard interpretation, and whole-theory agreement for W. If the emitted command sequence is semantically unsatisfiable, then*

$$\mathsf{Unsat}_{\mathsf{SMT}}(C_{\mathsf{emit}}) \quad \Longrightarrow \quad \mathsf{Unsat}^{\mathcal{D}}_{\mathsf{HO}}(\Phi_W).$$

The theorem ***`CommandEquivalence.unsat_source_script`*** *also admits the leading logic-selection command returned by* ***`buildScript`****.* ***`FactCommandRepresentation`*** *is the single-fact specialization of this aggregate path.* [73]

11.3 Operational trust status

`TranslateState.status` is `proved` exactly when its trust-reason array is empty; otherwise it retains the nonempty reasons in a `trusted` constructor. This operational classification is not itself a semantic representation witness. The Crush translator's extensible `emitTerm` route is marked trusted at its root, and unrestricted handlers, constants or closures without semantic proofs, and direct higher-order emission retain additional reasons. Individually proved components become a whole-run theorem only through the exact aggregate agreement described above. [74]

12 Proof architecture and exact scope

Contents of this section. The composed aggregate theorem; modeled translator behavior; explicit external boundaries.

Figure 1 summarizes the dependency chain. Solid arrows are backed by Lean theorems; the dashed reification arrow is structural, not denotational.

[72] Lean formalization: `Crush/Metatheory/VCG/CommandEquivalence.lean`.
[73] Lean formalization: `Crush/Metatheory/VCG/CommandEquivalence.lean`.
[74] Lean source: `Crush/Metatheory/VCG/Trust.lean`, `Crush/Translation/Monad.lean`, and `Crush/Metatheory/VCG/CommandEquivalence.lean`.

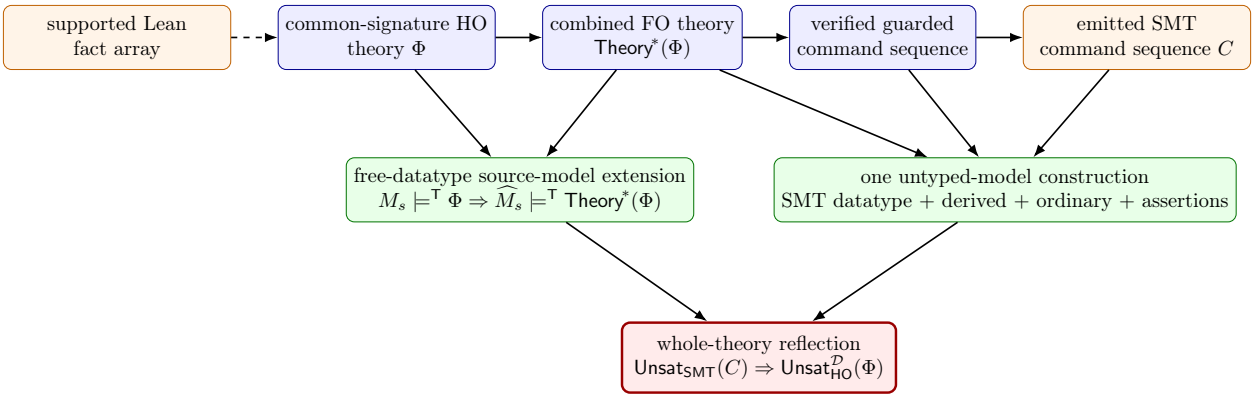

Figure 1: The current aggregate proof architecture. The free-datatype source-model condition and a uniform guard interpretation parameterize the model construction. Command-set agreement models translator reordering and duplicate suppression exactly in the membership-based untyped command semantics.

12.1 What the composed theorem proves

1. HO and FO syntax is intrinsically typed, and flattened defunctionalization is total on the complete modeled core.

2. Every source fact contributes its translated body, closure equations, extensionality formulas, and declaration occurrences to one combined target theory over one signature.

3. One canonical target model simultaneously satisfies that whole target theory whenever one source model satisfies the whole source theory.

4. Productive monomorphic datatype blocks have finite constructor-tree source semantics and exact SMT datatype command semantics in the untyped model. Nonproductive raw declarations are rejected by the same executable check used at the semantic boundary, and represented datatype prefixes have a standard-model witness.

5. Guarded-subset representations preserve typed terms, equality, and both quantifiers; recursive datatype guards select exactly the values that represent well-formed source datatype values.

6. One induced untyped SMT model validates SMT datatype commands, simultaneous guard definitions, ordinary declarations, and every guarded assertion.

7. Exact representation is semantic command-set equivalence. It therefore admits translator interleaving and duplicate elimination without weakening or strengthening the model obligations.

8. Common-environment reification retains the exact ordered Lean fact array, and the pure finite-theory VCG proves the meaning of its command sequence.

9. Command locations reconnect SMT datatype and recursive guard commands to the emitted sequence. `CommandEquivalence.build?` checks the complete sequence and returns existing guarded-representation evidence.

10. Semantic unsatisfiability of the agreed emitted command sequence reflects to unsatisfiability of the entire reified higher-order theory in every source model satisfying the free-datatype condition.

12.2 Exact boundaries

- No denotational semantics is postulated for arbitrary `Lean.Expr`; the final proposition concerns the reified higher-order theory.
- Semantic command unsatisfiability, including explicit evidence that every command is in the modeled fragment, is a premise. Correctness of an external solver's reported `unsat` answer requires replaying a checked solver proof or another independently checked argument.
- The extensible Crush translator does not automatically construct the shared encoding, SMT datatype representation, static guard representation, uniform guard interpretation, or common aggregate reification witness. Command reflection applies once those proof objects and exact agreement are present.
- `TranslateState.status` is operational metadata; it cannot replace a `TheoryRepresentation` or `SMT.GuardedTheoryRepresentation` witness.
- The translator's allocator is finite, while the formal `SMT.Encoding` interface supplies total injections over the modeled symbol family. The current theorem takes that total encoding as an explicit representation premise rather than claiming it is extracted from every allocator state.
- A ground datatype instance such as `Tree (A -> A)` is not accepted as a represented SMT datatype. The translator's acceptance check rejects the arrow-valued field, and certified reification reports `functionField`. The translator may continue by declaring the whole tree type as an opaque sort and its operations as ordinary uninterpreted symbols; a projected function value may still use the ordinary higher-order encoding. This fallback supplies none of the free-datatype laws and does not produce the represented SMT datatype block required by the command-equivalence theorem.
- The finite-array lowering emits a nonrecursive well-formedness `define-fun`. That command now has a nonvacuous graph-equation semantics. The current whole-pipeline representation does not yet prove the SMT array/select model construction or include this definition in its certified derived-command segment. Consequently the final exact command-set comparison rejects an array-bearing script rather than applying the reflection theorem to it.
- Unsupported dependent syntax, indexed or nonproductive inductives, dependent datatype fields, unsafe indirect recursion, unrestricted translation handlers, and direct higher-order commands remain outside the modeled theorem.

13 Machine-checked theorem inventory

Contents of this section. The named results that implement the proof story.

The principal declarations, in proof order, are:

1. `Datatype.ctor_inj`, `Datatype.ctor_ne`, `Datatype.ctor_cases`, `Datatype.sel_ctor`, `Datatype.test_ctor`, and `Datatype.Args.get_height_lt_ctor`: finite constructor-tree datatype laws.
2. `Datatype.IsFreeDatatypeModel.productive`: any source datatype carrier satisfying the free-datatype model condition supplies a productive typed block.
3. `Flattened.LambdaBody.denote_etaBody` and `Flattened.TargetArguments.completeApp_denote`: eta opening and one flattened application preserve repeated source application.

4. `Flattened.translate_denote`: the translated term component has the source denotation in the canonical model.
5. `Flattened.generatedFormulas_valid`: all generated closure equations and extensionality formulas are valid in that same model.
6. `Flattened.model_extension_theory`: one source model satisfying a finite theory extends to the combined flattened target theory.
7. `Flattened.target_theories_unsat_implies_source_unsat`: combined FO unsatisfiability reflects to the complete finite HO theory.
8. `SMT.encode_theories`: the pure encoder gives an exact semantic command-set representation of the combined theory.
9. `SMT.Datatype.command_sound` and `SMT.Datatype.EnvRepresentation.datatypeCommands_valid`: one block and the complete dependency-ordered SMT datatype prefix satisfy the raw datatype laws.
10. `SMT.Datatype.Representation.standardModel_exists` and `SMT.Datatype.EnvRepresentation.standardModel_exists`: represented datatype declarations and dependency-ordered prefixes have standard SMT models under the explicit source and integer interpretations.
11. `FO.FamilyTerm.guardDenote_rel`: related FO models preserve every term under guard-restricted quantifier semantics.
12. `Datatype.Val.wf_iff_selWF`: the structural datatype image equals the tester/selector predicate emitted by the Crush translator.
13. `SMT.Datatype.wfDefs_valid`: exact mutual `wf_T` definitions satisfy the untyped simultaneous graph semantics.
14. `SMT.guardTerm_rel_eval`: guard-aware untyped evaluation agrees with denotation in any symbol-related target model.
15. `SMT.guarded_lift`: SMT datatype commands, derived definitions, ordinary declarations, and guarded assertions hold in one untyped SMT model.
16. `SMT.representation_sound`: the unguarded complete representation has the corresponding one-model lifting theorem.
17. `SMT.commands_unsat_implies_source_theory_unsat`: semantic untyped-command unsatisfiability reflects to the reified higher-order theory under the free-datatype source-model condition.
18. `Reification.reifyTheory?`: an exact Lean fact array reifies under one datatype environment and reified signature without reordering.
19. `VCG.theoryCommands_represents` and `VCG.guardedTheoryCommands_represents`: pure aggregate generation retains ordinary and guarded representation evidence.
20. `VCG.theoryCommands_unsat_implies_source_unsat`: unsatisfiability of the pure finite-theory command sequence reflects to source-theory unsatisfiability.
21. `VCG.FactCommandRepresentation.unsat_source`: one translated fact's datatype and guard evidence reflects its exactly represented source sentence.
22. `VCG.CommandEquivalence.unsat_source` and `VCG.CommandEquivalence.unsat_source_script`: the emitted command sequence, with an optional leading logic command, reflects through checked whole-theory agreement.

The earlier unary results—`fundamental`, `fundamental_sentence`, and the unary `target_unsat_implies_source_unsat`—remain machine-checked as an independent reference model.